

WEEK 11 · REFERENCE

THE DATA LAYER, IN DEPTH

Every lecture slide the cycles replaced, the embedded demos that compute their own claims, and four measured answers to questions the cycles raise and do not stop for.

WHAT IS IN HERE

The storage half, the JSON half, `fetch` and CORS, `async` and the race — and the storage layer the machine wrote.

HOW TO READ IT

Each slide is the depth behind one cycle. The chip in the corner says which spine slide points here.

WHEN TO OPEN IT

When a reload shows something you did not expect, when the console says CORS — or before week 13, to see every mistake once more.

► **OPTIONAL DEPTH, NOT EXAMINED.** Everything the class assignment, the project and the exam need was typed in a cycle.

IT IS STILL THERE TOMORROW

Week 10 gave the application a shape.

This half is where the data stops living in the tab.

WHERE WE ARE

An application that is correct, architecturally clean — and that forgets everything when you close it.

AFTER THIS HALF YOU CAN

Write a storage layer that survives a reload, a corrupt key, and a change to your own data shape.

WHAT THIS UNLOCKS

Your project stops being a demo. The fetch half of this deck reads from a server through the same kind of layer.

The browser will keep exactly one thing for you: a string.



THE SAME APPLICATION TWICE. ONE OF THEM WRITES IT DOWN.

left: in memory · right: the same file, six lines longer

reload open

THE SAME APPLICATION TWICE. ONE OF THEM WRITES THE COLLECTION DOWN.

add an item to both remove the first from both reload both wipe the saved copy

IN MEMORY — WEEK 10'S APPLICATION

0 items · in memory only

WRITTEN DOWN — THE SAME APPLICATION, SIX LINES LONGER

0 items · saved under "demo01.pantry.v1"

Add a few items, then press "reload both".

The two panels run the same file. The lines that mention localStorage or JSON in it: 3. That is the whole of today's difference, and by the end of the lecture it lives in a file of its own.

► **ADD THREE ITEMS TO BOTH, AND DECIDE WHICH ONE IS BETTER WRITTEN.** You cannot tell. Then press "reload both".

WHAT THE BROWSER IS ACTUALLY WILLING TO KEEP

```
1 localStorage.setItem('key', 'a string'); // ← the only thing it takes
2
3 localStorage.getItem('key');           // → 'a string', or null
4 localStorage.getItem('never-written'); // → null
```

STRINGS IN

Not objects, not arrays, not numbers. Anything else is coerced.

STRINGS OUT

Or `null`, which is an answer and not a failure.

SO STORAGE IS TWO JOBS

Turn an object into a string safely.
Turn a string that might be **anything** back into an object safely.

TWO MISTAKES, BOTH SILENT, BOTH IN THE FIRST HOUR

left: setItem with an object · right: the very first read

↻ reload

open ↗

TWO MISTAKES, BOTH SILENT, BOTH IN THE FIRST HOUR

WRITING — SETITEM TAKES A STRING

store the object

store JSON.stringify(object)

WHAT WAS STORED

WHAT IS ON DISK

BYTES

READING — THE FIRST VISIT

the unguarded read

the honest read

```
const raw = localStorage.getItem('demo05.never-written')
raw        → null   (typeof object)
JSON.parse(raw) → null → IT DID NOT THROW
```

```
let items = JSON.parse(raw)
items.map(row) → ?
               → TypeError: Cannot read properties of null (reading 'map')
```

The parse was fine. The crash is four lines later, in render, and it says nothing about storage — which is why the next half hour goes into render.

Neither of these is an error the browser will tell you about. The first produces a string that is the same for every object in the world; the second produces null, which is a perfectly good value right up to the line that treats it as an array.

► **STORE THREE DIFFERENT OBJECTS.** Three went in. How many distinct strings came out?

THREE PLACES TO PUT A STRING. ONE SLIDE.

	LOCALSTORAGE	SESSIONSTORAGE	A COOKIE
survives a reload	yes	yes	until <code>max-age</code>
survives closing the browser	yes	no	until <code>max-age</code>
shared with another tab	yes	no — per tab	yes
sent to the server	no	no	on every single request
room	~5MB	~5MB	~4KB, refused in silence

The row that decides it is “sent to the server”.

HOW MUCH ROOM IS THERE, ACTUALLY

write 64KB blocks until it says no, and read the error

↶ reload

open ↷

FILL IT UNTIL IT SAYS NO

fill localStorage

clean up

this origin is currently holding 0.8 KB

Nothing written yet.

SAVE() WITH AN EMPTY CATCH

```
function save(items) {  
  try {  
    localStorage.setItem(KEY, JSON.stringify(items));  
  } catch {}  
}
```

the application says nothing

SAVE() THAT RETURNS WHAT HAPPENED

```
function save(state) {  
  try {  
    localStorage.setItem(KEY, JSON.stringify(payload));  
    return 'saved';  
  } catch (error) {  
    console.warn('[storage] write failed', error);  
    return 'full';  
  }  
}
```

the application says nothing yet either

The quota belongs to the **origin**, not to this page — so what was measured is what was left over after every other project you have run on this host and port. Two laptops in the same room give two different numbers, and both are right.

► The name is `QuotaExceededError` here and `NS_ERROR_DOM_QUOTA_REACHED` in older Firefox.

BRANCH ON "THE WRITE FAILED", NEVER ON THE NAME.

WHAT LOCALSTORAGE IS NOT

NOT A DATABASE

No queries, no indexes, no transactions. You read the **whole thing** and you write the **whole thing**.

NOT ASYNCHRONOUS

Every read and every write blocks the main thread. That is fine for a few hundred items, and it is what “What a keystroke costs” measures.

NOT SECURE

Every script on the page can read all of it. **No passwords, no tokens**, no anything you would mind a stranger seeing.

It is a drawer, and the whole drawer comes out every time.

WORKED EXAMPLE · 1 OF 2

BUILD THE LAYER, AND WATCH THE DRAWER

- 1 New file `js/storage.js`. One key, `<app>:v1` — the application's name and the version of the shape.
- 2 `save(items)` — `JSON.stringify(items)`: **the bare array**, nothing wrapped around it.
- 3 `load()` — and the first version, deliberately unguarded.
- 4 `app.js`: `hydrate(load())`, and **one added line**, `subscribe(persist)`, beside the `subscribe(render)` that was already there.
- 5 **DevTools > Application > Local Storage.**
Add an item, and watch the key appear. Add another, and watch the value change.

► Then reload. **THEN DELETE ONE CHARACTER FROM THE VALUE, BY HAND, AND RELOAD AGAIN.**

WHAT A KEYSTROKE COSTS

one word typed into a search box, three saving strategies ↶ reload open ↷

ONE WORD TYPED INTO A SEARCH BOX, THREE SAVING STRATEGIES

search type "פפריקה" for me add an item (a real change) close the tab NOW reset

STRATEGY	WRITES	TOTAL MS BLOCKING THE MAIN THREAD	ON DISK RIGHT NOW	CORRECT?
every change	0	0.0	2 items	yes
identity guard	0	0.0	2 items	yes
debounce 300ms	0	0.0	2 items	yes

Type something, or press the button, and watch the first two columns separate.

localStorage is **synchronous**. The millisecond column is time the page spent unable to do anything else — not time in the background.

▶ `subscribe(persist)` is correct — and after week 10, **TYPING IS A STATE CHANGE**. Press the button and watch the first column climb.

TWO FIXES, AND BOTH ARE RIGHT

THE IDENTITY GUARD

```
let lastSaved = null;

export function persist(state) {
  if (state.items === lastSaved) return;
  const result = save(state.items);
  if (result.ok) lastSaved = state.items;
}
```

Exact, and free. It works because week 10's handlers build new arrays instead of editing the one in state.

THE DEBOUNCE

```
let timer;
const saveSoon = (state) => {
  clearTimeout(timer);
  timer = setTimeout(() => save(state.items), 300);
};

addEventListener('pagehide', flush); // ← not optional
```

The general tool. **pagehide** closes the window in which the change is on screen and not on disk.

WHAT BELONGS IN STATE, AND WHAT BELONGS ON DISK

STATE — SIX FIELDS

- 1 items — **the collection**
- 2 query, category, sort
- 3 editingId — which row is open
- 4 storageNote — the sentence when a read or a write went wrong

DISK — ONE

items.

Restore a filter the user forgot they set and they see one row and no explanation — **a bug that reads exactly like “my data disappeared”**.

Those are two different questions.

THE LETTER OF CRUD THAT WAS MISSING

C, R, D — WEEKS 9 AND 10

The form adds, `render` reads, the row button removes.

U — THIS WEEK

One form, two jobs. `editingId` in state decides which; the submit handler is one `if`.

AND THE RULE IT FORCES

Data is drawn from state. A control's own value is not.

```
1  setState({  
2    items: state.items.map((item) => (item.id === editing.id ? { ...item, ...fields } : item)),  
3    editingId: null,  
4  });
```

WORKED EXAMPLE · 2 OF 2

GUARD IT, AND FINISH THE CYCLE

- 1 Go back to the unguarded `load()` from the first block and **fix it**: the access, the `null`, the parse, the shape.
- 2 Corrupt the key by hand in **Application** after each step, and reload. Four payloads, four different survivals.
- 3 The edit button, and the submit handler's one `if`.
- 4 “Clear all” — and **reload straight afterwards**.

► Keep **APPLICATION** open the whole time. Every change on screen should have a visible twin in the drawer.

THE QUOTA, MEASURED — AND WHY `save` RETURNS A SENTENCE

WHAT THE PROBE DID

```
1 const chunk = 'x'.repeat(512 * 1024); // 0.5 M character
2 for (let i = 0; i < 40; i++) {
3   localStorage.setItem('q' + i, chunk);
4 }
5 // Chromium 151: writes 1-9 succeed, write 10 throws
6 // QuotaExceededError (a DOMException, code 22)
```

SO THE LAYER SAYS SO

```
1 export function save(items) {
2   try {
3     localStorage.setItem(KEY, JSON.stringify(items));
4     return { ok: true, reason: null };
5   } catch (error) {
6     return { ok: false, reason: 'Not saved: storage is full.' };
7   }
8 }
```

Nine half-million-character writes fit, the tenth does not. **Per origin:** every Live Server project on `127.0.0.1` shares **about 5 million UTF-16 characters**, for the whole origin. Your write can fail because of somebody else's key.

► MDN lists one exception for `setItem`: `QuotaExceededError`. Older Firefox called it `NS_ERROR_DOM_QUOTA_REACHED` — so the catch never depends on the name.

JSON IS A TEXT FORMAT, AND IT HAS FIVE RULES

JAVASCRIPT

```
{  
  id: 'a1',  
  title: 'Whole-wheat flour',  
  number: 2,  
  fresh: true,  
}
```

JSON

```
{  
  "id": "a1",  
  "title": "Whole-wheat flour",  
  "number": 2,  
  "fresh": true  
}
```

1

Keys must be **strings**

2

Double quotes only

3

No functions

4

No comments

5

No trailing commas

THE ROUND TRIP, FIELD BY FIELD

the object · the string · what came back, compared

↺ reload

open ↗

AN OBJECT OUT THROUGH STRINGIFY, BACK THROUGH PARSE, AND COMPARED FIELD BY FIELD

THE OBJECT (EDITABLE)

JavaScript, evaluated on every keystroke

```
((
  id: 'a1',
  title: 'קמח חלא',
  number: 2,
  fresh: true,
  note: null,
  tags: ['בסיס', 'יבשים'],
  shelf: { room: 'חטבת', height: 3 },
  created: new Date('2026-09-01T08:00:00Z'),
  restock: undefined
))
```

THE STRING LOCALSTORAGE WOULD HOLD

```
{
  "id": "a1",
  "title": "קמח חלא",
  "number": 2,
  "fresh": true,
  "note": null,
  "tags": [
    "בסיס",
    "יבשים"
  ],
  "shelf": {
    "room": "חטבת",
    "height": 3
  },
  "created": "2026-09-01T08:00:00Z",
  "restock": null
}
```

276 bytes — this string is all localStorage ever holds

FIELD	BEFORE — VALUE AND TYPEOF	AFTER — VALUE AND TYPEOF	SURVIVED?
id	"a1" · string	"a1" · string	yes
title	"קמח חלא" · string	"קמח חלא" · string	yes
number	2 · number	2 · number	yes
fresh	true · boolean	true · boolean	yes
note	null · null	null · null	yes
tags	["בסיס", "יבשים"] · object	["בסיס", "יבשים"] · object	yes
shelf	{ "room": "חטבת", "height": 3 } · object	{ "room": "חטבת", "height": 3 } · object	yes
created	Date(2026-09-01T08:00:00.000Z) · object	"2026-09-01T08:00:00.000Z" · string	NO
restock	undefined · undefined	— gone —	NO

► Read the created row on its own. **WHAT TYPE WENT IN? WHAT CAME BACK?**

← SPINE · 14

TWELVE VALUES, THREE DIFFERENT ANSWERS EACH

alone · as a property · as an array member — real return values

↻ reload

open ↗

TWELVE VALUES, THREE WAYS. EVERY CELL IS A REAL RETURN VALUE.

THE VALUE	STRINGIFY(V)	STRINGIFY({ V })	STRINGIFY([V])	WHAT HAPPENED
a string	"mnp"	{"v":"mnp"}	["mnp"]	kept
a number	2	{"v":2}	[2]	kept
true	true	{"v":true}	[true]	kept
null	null	{"v":null}	[null]	kept
undefined	undefined (not a string!)	{}	[null]	DROPPED as a property, null in an array
a function	undefined (not a string!)	{}	[null]	DROPPED as a property, null in an array
a Symbol	undefined (not a string!)	{}	[null]	DROPPED as a property, null in an array
NaN	null	{"v":null}	[null]	becomes null — the number is gone, quietly
Infinity	null	{"v":null}	[null]	becomes null — same
a Date	"2026-09-01T08:00:00.000Z"	{"v":"2026-09-01T08:00:00.000Z"}	["2026-09-01T08:00:00.000Z"]	becomes a STRING, via Date.toJSON
a Map	{}	{"v":{}}	[{}]	becomes {} — the entries are gone
a Set	{}	{"v":{}}	[{}]	becomes {} — the members are gone
a BigInt	throws TypeError	throws TypeError	throws TypeError	THROWS — TypeError
a circular object	throws TypeError	throws TypeError	throws TypeError	THROWS — TypeError

8 of these 14 values change meaning WITHOUT SAYING SO. 2 of them throw, and those are the ones you will fix today rather than in March.

The two rows that **throw** are the lucky ones. Everything else on this table fails silently, which means it fails later, somewhere else, in a function that has nothing to do with storage.

► **COVER THE LAST COLUMN AND PREDICT THE FUNCTION ROW.** Then look at the two rows that **THROW** — those are the lucky ones.

THE DATE, AND THE CRASH THAT ARRIVES TOMORROW

WHAT EVERYBODY WRITES

```
1 const item = { created: new Date() };
2
3 save(item);
4 // ... tomorrow, after a reload ...
5 item.created.getFullYear();
6 // TypeError: created.getFullYear
7 //           is not a function
```

Works perfectly. Until the first reload.

THE RULE

```
1 const item = { created: new Date().toISOString() };
2
3 // and where you display it, and nowhere else:
4 new Date(item.created).toLocaleDateString('he-IL');
```

Store the string. Convert at the point of use.

THE SECOND ARGUMENT, AND WHAT IT COSTS

replacer as an array · as a function · and the reviver

reload open

STRINGIFY(VALUE, REPLACER) AND PARSE(TEXT, REVIVER)

no replacer · replacer as an array — an allowlist · replacer as a function · reviver — bring the Date back

THE CALL
JSON.stringify(item, null, 2)
JSON.parse(text)

WHAT WAS WRITTEN

```
{
  "id": "a1",
  "title": "אלקתן חלק",
  "number": 2,
  "secret": "do-not-save-me",
  "created": "2026-09-01T08:00:00.000Z",
  "shelf": {
    "room": "חטבת",
    "height": 3
  }
}
```

188 bytes

WHAT CAME BACK — VALUE AND TYPEOF
id: "a1" · string
title: "אלקתן חלק" · string
number: 2 · number
secret: "do-not-save-me" · string
created: "2026-09-01T08:00:00.000Z" · string
shelf: room: "חטבת" · string
height: 3 · number

The Date is a string on the way back, and `secret` went to disk with everything else.

► The reviver gets the Date back — by **GUESSING FROM THE SHAPE OF A STRING**. Consider what happens the day somebody stores a note that looks like a timestamp.

READING A NESTED STRUCTURE YOU DID NOT WRITE

OPTIMISTIC

```
const city = data.user.address.city;  
const first = data.items[0].title;
```

Two `TypeError` crashes waiting for a payload that is one field short.

DEFENSIVE AT THE EDGE

```
const city = data?.user?.address?.city ?? 'unknown';  
const first = data?.items?.[0]?.title ?? null;
```

Good for reading. Not a substitute for checking the shape — see “Parse, then check”.

? . stops a crash. It does not tell you the data is right.

AN EMPTY CATCH IS WORSE THAN NO CATCH

SILENT

```
function save(items) {  
  try {  
    localStorage.setItem(KEY, JSON.stringify(items));  
  } catch {}  
}
```

The user has an application that **looks exactly like one that is saving**. They find out when they close the tab.

IT SAYS WHAT HAPPENED

```
function save(items) {  
  try {  
    localStorage.setItem(KEY, JSON.stringify(items));  
    return { ok: true, reason: null };  
  } catch (error) {  
    console.warn('[storage] write failed', error);  
    return { ok: false, reason: 'Not saved.' };  
  }  
}
```

As a result the UI can draw a sentence from — the shape the class assignment's `save` returns.

SIX THINGS THAT ARE NOT YOUR DATA

the same six payloads through two loaders

↺ reload

open ↗

THE SAME SIX PAYLOADS THROUGH TWO LOADERS

1 · truncated

2 · nothing saved

3 · the string null

4 · valid JSON, wrong shape

5 · one bad member

6 · a version from the future

0 · perfectly fine

WHAT IS ON DISK

```
{ "v": 1, "items": [{"id": "a1", "tit
```

THE LOADER EVERYBODY WRITES

```
function load() {  
  const raw = localStorage.getItem(KEY);  
  return JSON.parse(raw).items;  
}
```

THREW SyntaxError: Unterminated string in JSON at position 31 (line 1 column 32)

THE LOADER IN THE ASSIGNMENT

```
function load() {  
  let raw;  
  try { raw = localStorage.getItem(KEY); } catch { return null; }  
  if (raw === null) return null; // 2  
  let parsed;  
  try { parsed = JSON.parse(raw); } catch { return null; } // 1  
  return migrate(parsed); // 3, 4, 5, 6  
}
```

returned null — the caller seeds instead, and the page draws

נתוני דוגמה

1

payloads run: 1 · the naive loader produced a usable collection 0 of those times.

► **RUN 1 AND 2 AND PROPOSE THE FIXES.** Write both down. Then run 3.

PARSE, THEN CHECK — AND CHECK ONLY WHAT YOU READ

```
1  const isItem = (value) =>
2    value !== null &&
3    typeof value === 'object' &&
4    typeof value.id === 'string' && value.id !== '' &&
5    typeof value.title === 'string' &&
6    Number.isFinite(value.number) &&
7    typeof value.category === 'string';
8
9  if (!Array.isArray(parsed)) return { items: [], reason: 'Saved data was not a list.' };
10 const items = parsed.filter(isItem); // keep the good ones, drop the rest
```

► **THE FOUR FIELDS THE APPLICATION ACTUALLY READS, WITH THE TYPES IT ACTUALLY ASSUMES. NOTHING MORE.** A validator that mirrors the whole schema is a second definition of the item.

“CLEAR ALL” IS SAVING AN EMPTY ARRAY

CLEARING IS SAVING

```
clear.addEventListener('click', () => {  
  if (!confirm('Delete all?')) return;  
  setState({ items: [] });  
  // persist writes '[]' under library:v1  
});
```

Reload: an empty collection, and the disk says “**the user emptied it**”.

AND REMOVEITEM ?

```
localStorage.removeItem('library:v1');  
// reload: getItem returns null  
// load() returns { items: [], reason: null }
```

Also empty. Under `<app>:v1` an absent key **is** an empty collection. The famous “demo data comes back” bug lived only in applications that **seed on an empty read**.

That is why the course never seeds on boot: it would hide any fault that makes the read come back empty.

THE SHAPE CHANGED. THEIR BROWSERS DID NOT.

MONDAY

You ship. People use it. Their data is written in **version 1**.

FRIDAY

An item gains a field. Your code reads it everywhere.

FRIDAY EVENING

Every one of them still holds version 1 — and it is **the only copy of their data there is**.

**A version costs three characters,
and it is written before you need it.**

► The project and the class assignment put it in the key: a bare array under `<app>:v1`, and a new shape reads `<app>:v2`. An envelope with a `v` field is the other pattern — homework 4's cache uses it.

YESTERDAY'S DATA, THREE PROGRAMS

crashes · silently wrong · migrated

reload open

THE SHAPE CHANGED. THEIR BROWSERS DID NOT.

put v0 on disk — a bare array put v1 on disk — no tags field put v2 on disk — the current shape

WHAT IS ON DISK

```
{ "v": 1, "items": [ { "id": "a1", "title": "קמח מלא", "number": 2, "category": "יבשים" }, { "id": "a2", "title": "פפריקה מעושנת", "number": 3, "category": "תבלינים" } ] }
```

V2 CODE, NO MIGRATION

```
items.map((i) => i.tags.length)
```

THREW `TypeError: Cannot read properties of undefined (reading 'length')`

V2 CODE, GUARDED WITH ?.

```
items.map((i) => i.tags?.length ?? 0)
```

קמח מלא	0 tags
פפריקה מעושנת	0 tags

No error, no blank page, and 2 of 2 rows are WRONG. Nothing on this screen says so, and nothing ever will.

V2 CODE, WITH A MIGRATION

```
if (data.v === 1) {  
  items = data.items.map(  
    (i) => ({ ...i, tags: [i.category] });  
  );  
  save({ v: 2, items }); // once  
}
```

קמח מלא	1 tags
פפריקה מעושנת	1 tags

v1 → v2

on disk now: { "v": 2, "items": [{ "id": "a1", "title": "קמח מלא", "number": 2, "cat... — the migration ran ONCE. Reload and the v1 branch is not taken.

A version number costs one field and is written before you need it. The alternative is finding out from a user which of three shapes their browser is holding.

► **STAY ON THE MIDDLE PANEL FOR A MINUTE.** No error, no blank page, and every row is wrong. (The demo stores an envelope, { v, items } — homework 4's pattern.)

THREE ANSWERS, NOT TWO

THE ENVELOPE PATTERN — HOMEWORK 4'S CACHE HAS THE SAME THREE ANSWERS

```
1 function migrate(parsed) {  
2   if (Array.isArray(parsed)) return parsed.filter(isItem);           // v0: adopt it  
3  
4   if (parsed?.v === VERSION) return parsed.items.filter(isItem);    // current  
5  
6   // A version we have never heard of. Data written by a NEWER version may mean  
7   // something different in every field — a guess CORRUPTS rather than loses.  
8   return null;  
9 }
```

► Version 0 is **NOT HYPOTHETICAL**: it is whatever your code wrote before you thought about versions. In homework 4 it is a bare map of cities, and `migrate` adopts it. The project needs none of this — its key's `:v1` is the version.

WHAT `JSON.parse` DOES WITH NOTHING

THE CALL

WHAT COMES BACK — CHROMIUM 151

`localStorage.getItem('missing')` → the key was never written

`JSON.parse(null)` `null` — **no throw**: `null` becomes the string `"null"`, which is valid JSON

`JSON.parse(undefined)` `SyntaxError: "undefined" is not valid JSON`

`JSON.parse('')` `SyntaxError: Unexpected end of JSON input`

```
1 if (raw === null) return { items: [], reason: null }; // BEFORE the parse
2 let parsed;
3 try { parsed = JSON.parse(raw); } catch { return { items: [], reason: 'Unreadable.' }; }
4 if (!Array.isArray(parsed)) return { items: [], reason: 'Not a list.' }; // AFTER
```

► The first visit never reaches the `catch`. Test `raw === null` **BEFORE** parsing, and `Array.isArray` **AFTER**.

THE WHOLE OF `fetch`, AND THEN THE HARD PART

```
1 const res = await fetch(url);           // a Response, not the data
2
3 if (!res.ok) { /* the server answered, and said no */ }
4
5 const data = await res.json();           // a SECOND await, a second failure
```

TWO LINES OF API

Send a request, read a body. That is the whole surface.

THREE LINES OF JUDGEMENT

Did it complete? Did the server agree? Was the body what you expected?

AND THE DEFAULT IS WRONG

No timeout. No status check. No shape check. **All three are yours.**

THE LAYER, AND WHAT IS NOT IN IT

```
1 // api.js – LAYER 5. No document. No setState. No decision about the screen.
2
3 export async function searchTitles(query, { signal }) {
4   const url = new URL(ENDPOINT);
5   url.searchParams.set('q', query);      // NOT '&q=' + query
6
7   const res = await fetch(url, { signal });
8   if (!res.ok) throw new ApiError('status', `The catalogue answered ${res.status}.`, '');
9
10  let payload;
11  try { payload = await res.json(); }
12  catch { throw new ApiError('shape', 'The catalogue did not answer with JSON.', ''); }
13
14  return toSuggestions(payload);          // one normaliser, two sources
15 }
```

► It returns a collection, or it throws a **WHOLE SENTENCE**. Never a `null` that means “work it out”.

SENDING SOMETHING: POST, AND THE ONE HEADER THAT MATTERS

```
1 const res = await fetch(url, {  
2   method: 'POST',  
3   headers: { 'content-type': 'application/json' },  
4   body: JSON.stringify({ title, number }), // a STRING, always –  
5                                             // the same rule as setItem  
6   signal,  
7 });  
8 if (!res.ok) throw new ApiError('status', '...', `HTTP ${res.status}`);
```

body IS A STRING

Exactly like `localStorage.setItem`. Hand it an object and it coerces.

THE HEADER IS NOT DECORATION

Without it the server guesses, and most of them guess `text/plain`.

AND IT TRIGGERS A PREFLIGHT

A `content-type: application/json` is not a “simple” request. The browser asks permission first — the CORS slides next.

WORKED EXAMPLE · 1 OF 2

BUILD THE LAYER, AND BREAK IT ON PURPOSE

- 1 Write `searchTitles` against a real API. Watch it work.
- 2 Type the endpoint wrong. **Read the error out loud.**
- 3 DevTools › Network › throttle to **Slow 3G**. Type again.
- 4 Tick **Offline**. Type again.
- 5 Note what the user just saw. **Nothing. Twice.**



THE MOST CONFUSING ERROR IN FRONT-END DEVELOPMENT

WHAT YOUR CODE GETS

```
TypeError: Failed to fetch
```

No status. No headers. No body. No reason.

WHAT ACTUALLY HAPPENED

- 1 the request left your machine
- 2 the server received it
- 3 the server answered
200
- 4 the browser
refused to hand it to you

The server said yes. Your browser said no.

THE SAME CODE, TWO SERVERS

one Wikipedia search with origin=* sends the header · the same URL without it does not

↶ reload

open ↷

THE SAME CODE, TWO SERVERS

fetch with origin=* — the header comes back

the same URL without it — it does not

reset

THE CODE, AND IT IS IDENTICAL FOR BOTH

```
try {
  const res = await fetch(url);
  const data = await res.json();
} catch (err) {
  // what lands here?
}
```

ACCESS-CONTROL-ALLOW-ORIGIN: *

—

NO SUCH HEADER

—

WHAT HAPPENED

ALLOWED ORIGIN

BLOCKED ORIGIN

the request left your machine

yes

yes

the server processed it

yes

yes

the server answered 200

yes

yes

the Network tab shows the row

yes

yes

your JavaScript received the answer

yes

NO

what your catch was given

nothing — it succeeded

TypeError: Failed to fetch

► Press the second button **WITH THE NETWORK TAB OPEN**. The row is there. The status is 200. The size is real.

← SPINE · 19

WHY THE BROWSER DOES THIS AT ALL

THE RULE

A page may **send** a request anywhere. It may only **read** the answer if the answer says it may.

WHY

Otherwise any page you open could ask your bank for your statement — **using your cookies** — and read the reply.

SO

It is not aimed at you. You are standing in the way of something that is aimed at somebody worse.

► **CHECK BEFORE YOU BUILD.** Open the URL, then look for `access-control-allow-origin` in the Network tab. Two minutes.

THERE IS NO TIMEOUT, AND THERE NEVER HAS BEEN

a server that takes eight seconds, and four ways to react

reloadopen

THE SERVER TAKES 8.0 SECONDS TO ANSWER

server delay ☐ never answer at all

no timeoutAbortSignal.timeout(2000)timeout + one retrycancel by handreset

LIVE

waiting for 0.00 s

idle

THE TWO WAYS TO WRITE A TIMEOUT

// short, and it cannot leak a timer
const signal = AbortSignal.any([
 caller.signal,
 AbortSignal.timeout(2000),
]);
await fetch(url, { signal });

// the long way, which is what an assistant writes
const c = new AbortController();
const t = setTimeout(() => c.abort(), 2000);
try { await fetch(url, { signal: c.signal }); }
finally { clearTimeout(t); } // <- the line people leave out

STRATEGY

SERVER DELAY

THE USER WAITED

ATTEMPTS

HOW IT ENDED

► Run **NO TIMEOUT** at eight seconds first, and watch what the user sees while it runs.

CORS IS A SERVER'S DECISION, AND IT CHANGES

THE REQUEST, FROM A PAGE ON ~~7 SEP 2026, CHROMIUM~~

he.wikipedia.org/w/api.php?blockedquery&list=failed&format=json

the same URL + `&origin=*` 200, readable — the header comes back

data.gov.il/api/3/action/datastore_search?access-control-allow-origin: *

- ▶ Wikipedia sends the header only to a caller who asks with `origin=*`. data.gov.il was this course's blocked example; **TODAY IT SENDS THE HEADER.** Neither change was in anybody's JavaScript on the page.

Check the header in the Network tab before you build on an API.

422 IS AN ANSWER, NOT “NOTHING FOUND”

OPEN LIBRARY, SEARCH.JSON?Q=AB

THE STATUSES YOU WILL MEET

```
1 const res = await fetch(url); // resolves: status 422
2 const body = await res.json(); // {"detail":[{"type":"value_error", ...
3 // "msg":"... Query too short, must be at least 3 characters"}]}
4 return body.docs ?? []; // [] — drawn as "nothing found"
```

fix the question

down, do not

hammer

500 / 503

their fault — say so

fetch resolved, res.ok is false, and the body is JSON. **Don't ask again at once gets the same answer** — the server already answered. The one exception is **429**: wait (for as long as its Retry-After header says, when it sends one), then ask again.

EVERYTHING ON YOUR SCREEN WAS TYPED BY YOU

WHAT WE HAVE

Whatever the user added — plus six demo items, if they pressed the button. Correct, persistent, and entirely invented.

WHAT A REAL APP HAS

A catalogue somebody else maintains, on a machine you do not control, over a network you cannot see.

AND WHAT THAT COSTS

It is **slow**. It **fails**. It sometimes answers **something else**. All three are now your problem.

Modern web apps are data-driven: data comes from a server, is stored in the browser, and is exchanged in a standard format.

ONE THREAD, AND WHAT HOLDING IT COSTS

two seconds, three ways – and a counter that says whether the page is alive↻ reloadopen ↗

TWO SECONDS, THREE WAYS

block for 2s

await 2s

setTimeout 2s

reset

THE PAGE, WHILE IT HAPPENS

frames painted since you pressed: **0**

A healthy page paints about 60 of these a second. Two seconds should be ~120.

type here during the two seconds

Nothing typed yet.

WHAT ACTUALLY RAN

RUN

HOW

WALL CLOCK

FRAMES PAINTED

THE THREAD WAS

► Press **BLOCK FOR 2S** and try to type in the box while it runs. Then press **AWAIT 2S** and do the same.

THE SAME JOB, THREE WAYS TO WRITE IT

find the user, then the orders, then the total – measured↺ reloadopen ↗

FIND THE USER, THEN THE ORDERS, THEN THE TOTAL

run all three ☐ make step 2 fail Ready.

CALLBACKS

```
findUserCb('דנה', (err, user) => {
  if (err) return show(err);
  findOrdersCb(user.id, (err, orders) => {
    if (err) return show(err);
    findTotalCb(orders, (err, total) => {
      if (err) return show(err);
      show(total);
    });
  });
});
```

—

PROMISE CHAIN

```
findUser('דנה')
  .then((user) => findOrders(user.id))
  .then((orders) => findTotal(orders))
  .then((total) => show(total))
  .catch(show);
```

—

ASYNC / AWAIT

```
try {
  const user = await findUser('דנה');
  const orders = await findOrders(user.id);
  const total = await findTotal(orders);
  show(total);
} catch (err) {
  show(err);
}
```

—

VERSION	ANSWER	WALL CLOCK	DEEPEST INDENT	PLACES THE FAILURE IS HANDLED

► Run it once with the failure off. Same answer, same time. Then tick **MAKE STEP 2 FAIL**.

← SPINE · 27

42

A PROMISE IS THREE STATES, AND IT SETTLES ONCE

```
1 const p = new Promise((resolve, reject) => {  
2   // pending until one of them is called  
3 });  
4  
5 p.then((value) => { /* fulfilled */ })  
6 .catch((error) => { /* rejected */ })  
7 .finally(() => { /* both paths */ });
```

1 pending
— nobody knows yet

2 fulfilled
— there is a value

3 rejected
— there is a reason

A settled promise never settles again.

try/catch WORKS AGAIN, AND THAT IS THE WHOLE GIFT

A CALLBACK CANNOT BE CAUGHT

```
1 try {  
2   load(url, (err, data) => {  
3     // by the time this runs, the try  
4     // below has ALREADY finished  
5   });  
6 } catch { /* never reached */ }
```

AN AWAIT CAN

```
1 try {  
2   const data = await load(url);  
3   show(data);  
4 } catch (error) {  
5   // a rejection ANYWHERE in the chain lands here  
6   showProblem(error);  
7 } finally {  
8   stopSpinner();           // both paths  
9 }
```

- `await` turns a rejection into a `throw` at exactly the line you were waiting on. That is the entire syntactic trick.

A REJECTION IS NOT AN EXCEPTION UNTIL SOMEBODY ASKS

the same rejected promise, caught now and caught a second later

↺ reload

open ↗

THE THREE STATES, AND THE FOURTH THING NOBODY MENTIONS

new promise

resolve it

reject it

state: —

WHAT RAN, IN ORDER

press "new promise"

THE CODE THAT IS ATTACHED

```
const p = new Promise((resolve, reject) => {  
  // nothing here settles it – the buttons do  
});  
  
p.then((value) => log('then: ' + value))  
  .catch((err) => log('catch: ' + err.message))  
  .finally(() => log('finally – on both paths'));
```

A REJECTION NOBODY IS LISTENING TO

reject, and catch it immediately

reject, and catch it one second later

unhandledrejection events so far: 0

CAUGHT IN THE SAME TICK

```
const p = Promise.reject(new Error('boom'));  
p.catch(handle); // attached NOW
```

—

CAUGHT ONE SECOND LATER

```
const p = Promise.reject(new Error('boom'));  
setTimeout(() => p.catch(handle), 1000);
```

—

Press both. The promise is identical; only the moment the listener arrives differs.

► Press both buttons at the bottom. **THE PROMISE IS IDENTICAL** — only the moment the listener arrives differs.

`async` AND `await` ARE THE SAME THING, WRITTEN FLAT

WHAT PEOPLE THINK IT MEANS

```
1 // "await runs this on another thread
2 // and waits for it there"
3
4 const items = getItem(); // items is a Promise
```

WHAT IT MEANS

```
1 // "suspend THIS function and give the
2 // one thread back until the answer is here"
3
4 const items = await getItem();
5 // ↑ an async function ALWAYS returns a Promise
```

► `items.length` without the `await` is undefined — and that is the single most common mistake in this subject.

WHAT ORDER DOES THIS ACTUALLY RUN IN?

seven statements, and the order is not the order

reload

open

SEVEN STATEMENTS. WHAT ORDER DO THEY RUN IN?

run it

starve the timer

Guess first. Then press.

THE SOURCE, AS WRITTEN

```
console.log('1 - synchronous, top');           // line 1

setTimeout(() => console.log('timer'), 0);      // line 3

Promise.resolve().then(() => console.log('promise .then')); // line 5

queueMicrotask(() => console.log('queueMicrotask')); // line 7

(async () => {
  console.log('inside async, BEFORE the await'); // line 10
  await null;                                   // line 11
  console.log('inside async, AFTER the await');  // line 12
})();

console.log('2 - synchronous, bottom');        // line 15
```

THE ORDER IT RAN

-

#	LABEL	QUEUE	WRITTEN AT LINE	RAN AT POSITION
---	-------	-------	-----------------	-----------------

A TIMER THAT ASKED FOR ZERO

The same measurement four times, with a longer chain of promise callbacks each time. In every run a `setTimeout(fn, 0)` is scheduled **first**, and then the chain is built. Microtasks drain completely before the next macrotask, and a microtask that queues another microtask joins the same drain.

► **DECIDE BEFORE YOU PRESS.** Write your answer down first.

← SPINE · 27

FIVE REQUESTS, AND THE LOOP THAT COSTS YOU FIVE TIMES

CORRECT, AND FIVE TIMES SLOWER

```
for (const url of urls) {  
  results.push(await fetch(url));  
}  
// each one STARTS after the last one finished
```

CORRECT

```
const results = await Promise.all(  
  urls.map((url) => fetch(url)),  
);  
// all five start now; you wait for the slowest
```

► The loop is **RIGHT** when step 2 needs step 1's answer. These five do not.

ALL · ALLSETTLED · RACE, WITH A STOPWATCH

five tasks – 500, 300, 200, 400, 100 ms↻ reloadopen ↗

FIVE TASKS: 500, 300, 200, 400, 100 MS

await in a loop **Promise.all** Promise.allSettled Promise.race ☐ make task 3 fail reset

WHAT EACH ONE RETURNED
–

THE SHAPE OF EACH CALL
for (const t of tasks) out.push(await t()); // sum of all five

await Promise.all(tasks.map(t => t())); // slowest one
await Promise.allSettled(tasks.map(t => t())); // slowest one, never rejects
await Promise.race(tasks.map(t => t())); // fastest one to SETTLE
–

CALL	WALL CLOCK	VS THE THEORETICAL BEST	OUTCOME	RESULTS YOU CAN ACTUALLY USE
------	------------	-------------------------	---------	------------------------------

► Run the loop and `Promise.all` back to back. Then tick **MAKE TASK 3 FAIL** and run `all` and `allSettled`.

WHICH ONE, AND WHEN

`Promise.all`

I need all of them. Three panels of a dashboard. One fails and there is nothing worth drawing.

`Promise.allSettled`

I need what I can get. Six independent widgets. Five that loaded beat a blank page.

`Promise.race`

Whichever settles first. Its real use is a timeout — and note *settles*, so a fast failure wins.

RETRY, AND WHAT IT ACTUALLY COSTS

```
1  for (let n = 0; n <= retries; n++) {  
2    try {  
3      return await attemptOnce(signal);  
4    } catch (error) {  
5      if (error.name === 'AbortError') throw error;  
6      if (error instanceof ApiError) throw error;  
7      if (n === retries) throw asApiError(error);  
8      await sleep(300 * 2 ** n, signal);  
9    }  
10 }
```

- 1 timeout, dropped connection**
— retry. It might work.
- 2 any status**
— never. The server already answered.
- 3 abort**
— never, and hard. They asked you to stop.

A retry does not make a slow request fast. It makes a failing one take twice as long to fail.

WORKED EXAMPLE · 2 OF 2

FIX THE BOX, IN THE ORDER THE BUGS APPEAR

- 1 Count the requests in the Network tab. **Six.** Add the debounce. **One.**
- 2 Throttle to Slow 3G and type fast. **The wrong list appears.** Add the abort.
- 3 Type fast again. **The panel flashes red.** Add the `AbortError` line.
- 4 Add the ticket, and work out what it catches that the abort does not.



TWO FAULTS IN A STORAGE LAYER THE MACHINE WROTE

```
1  const KEY = 'items';
2
3  export function load() {
4    try {
5      return JSON.parse(localStorage.getItem(KEY)) || [];
6    } catch (e) {
7      localStorage.removeItem(KEY);
8      return [];
9    }
10 }
11
12 export function save(items) {
13   try { localStorage.setItem(KEY, JSON.stringify(items)); } catch {}
14 }
```

► It runs. It passes every manual test. **FIND THE TWO THAT WILL COST A USER THEIR DATA** — and the third that will cost you an afternoon.

THREE QUESTIONS BEFORE YOU GO – THE STORAGE HALF

1

A user deletes every item and reloads.

What does `load()` return, and why is it never the demo data?

2

`JSON.parse` succeeded.

What do you now know about the data, and what do you not?

3

You add a fifth handler to `events.js` next month.

How many lines do you have to write for it to be saved?

WHAT THIS DECK IS, AND WHAT IT IS NOT

NOT EXAMINED

Nothing on these slides is graded on its own. The class assignment, the project and the exam ask only for what a cycle typed.

WHERE EACH TOPIC WAS TYPED

Storage layer — cycle 1. Corrupt data and the edit — cycle 2. `fetch` and the break — cycle 3. Five states and the race — cycle 4. The machine's storage layer — cycle 5.

THE ONE TEST THAT MATTERS

Corrupt your own key by hand and reload. If the page comes up, you are finished. **If it does not, you are not.**

► Week 12 puts types on the same layers. Nothing in this deck changes when it does.